Comparative Evaluation of GCN, GAT, and GPS Architectures for Node Classification on ogbn-arxiv

Emanuel Cortes Lugo
Department of Electrical and Computer Engineering
University of Central Florida
Orlando, FL, USA
emanuel.corteslugo@ucf.edu

Laith Yusuf
Department of Electrical and Computer Engineering
University of Central Florida
Orlando, FL, USA
la923018@ucf.edu

***Abstract*—Graph neural networks have become one of the most popular approaches for learning on graph-structured data, but it is not always clear which architecture works best in practice, especially on large graphs with limited compute. This paper compares three well-known GNN architectures—Graph Convolutional Network (GCN), Graph Attention Network (GAT), and the General, Powerful, Scalable graph transformer (GPS)—on the ogbn-arxiv node classification benchmark. The dataset has 169,343 computer-science paper nodes, 1,166,243 directed citation edges, and 40 subject-area classes. All models are evaluated using the official OGB train/validation/test split and top-1 accuracy. In our experiments, GCN achieved the highest test accuracy at 72.00%, GAT came in close at 71.75%, and GPS reached 68.97%. GPS underperformed mainly because mini-batch training with ClusterLoader limits how much global context each node can attend to. A per-class breakdown shows that GPS still helps on cross-domain categories, which partially supports the idea that global attention is useful for interdisciplinary topics.**

***Index Terms*—graph neural networks, graph convolutional network, graph attention network, graph transformer, GPS, node classification, ogbn-arxiv, Open Graph Benchmark, mini-batch training**

I. Introduction

Graph-structured data shows up in many different areas, including social networks, knowledge bases, molecular chemistry, and academic citation networks. These datasets all share the property that relationships between entities matter just as much as the entities themselves. Standard deep learning models like CNNs and MLPs are not designed to handle this kind of relational structure, which is why Graph Neural Networks (GNNs) were developed. GNNs work by passing messages between connected nodes, allowing each node to build up a representation that reflects both its own features and the features of its neighbors [1].

For this project, we use the ogbn-arxiv dataset, which is a commonly used large-scale benchmark for node classification. It represents 169,343 computer-science papers as nodes, connected by 1,166,243 directed citation edges. Each paper belongs to one of 40 arXiv subject categories, and each node has a 128-dimensional feature vector made from averaged word embeddings of the paper's title and abstract. One of the things that makes this dataset interesting is that some categories are large and self-contained (like cs.CV and cs.LG), while others are small and pull from many different areas (like cs.HC and cs.GL). This variety makes it a good testbed for comparing how different architectures handle diverse graph structure.

We evaluate three architectures that represent increasing levels of complexity. The first is GCN [1], which uses a fixed symmetric normalization to aggregate neighbor features and is simple and fast to train. The second is GAT [2], which adds learned attention weights so that each node can focus more on certain neighbors. The third is GPS [3], which combines local message passing with global multi-head self-attention and positional encodings, making it the most expressive of the three but also the most expensive to run.

The main questions we wanted to answer are: (1) which architecture gives the best accuracy-to-cost tradeoff under realistic compute constraints, and (2) does GPS's global attention mechanism specifically help categories that cite papers from many different fields?

Our main contributions in this work are: implementing and training all three models under the same framework and evaluation protocol, measuring the accuracy cost that mini-batch training imposes on GPS, and doing a per-class accuracy analysis to check whether global attention helps interdisciplinary categories.

II. Related Work

GNN research has grown a lot over the past several years, with many different architectures proposed for node classification and other graph tasks. For this project, we focus on three specific model families and the benchmark used to evaluate them.

A. Graph Convolutional Networks

Kipf and Welling [1] introduced GCN as a simple and scalable method for semi-supervised node classification. Their main idea was to approximate spectral graph convolutions using just the normalized adjacency matrix and a learnable

weight matrix at each layer. This keeps the model simple and fast while still capturing local neighborhood structure. They showed that even a two-layer GCN beats older methods on the Cora, Citeseer, and Pubmed benchmarks. One known issue with GCN is that stacking many layers causes over-smoothing, where all nodes end up with similar representations. Adding residual connections and normalization layers helps, but it is still something to keep in mind when designing deeper models.

B. Graph Attention Networks

Veličković et al. [2] proposed GAT to fix a limitation in GCN: every neighbor gets the same weight during aggregation, regardless of how relevant it is. GAT learns attention coefficients for each edge by comparing the representations of the two endpoints, so the model can focus on more important neighbors. It also uses multi-head attention, where several independent attention mechanisms run in parallel and their outputs are concatenated, which helps with stability during training. On standard citation network tasks, GAT performed as well as or better than GCN. Later versions of GAT improved how the attention scores are computed, though the original is still widely used as a baseline.

C. General, Powerful, Scalable Graph Transformers

Rampášek et al. [3] introduced GPS to address a fundamental limitation shared by both GCN and GAT: they can only aggregate information from nearby neighbors. If two nodes are far apart in the graph, information has to travel through many intermediate nodes, which can get lost or distorted along the way. GPS solves this by combining a standard local message-passing step with a global Transformer that lets every node attend to every other node in the graph at once. It also uses Laplacian Positional Encodings (LapPE) to give each node a sense of its structural position in the graph, since Transformers otherwise have no concept of where nodes are relative to each other. GPS reported around 75% test accuracy on ogbn-arxiv, which is noticeably higher than what standard message-passing models achieve. The main practical challenge is that the global attention step requires $\mathcal{O}(N^2)$ memory, which becomes a problem for large graphs.

D. Node Classification Benchmarks and the Open Graph Benchmark

The Open Graph Benchmark (OGB) was created to make GNN comparisons more fair and reproducible. Before OGB, most papers reported results on small citation graphs like Cora and Citeseer, where even simple models do well and it is hard to tell which architecture is actually better on harder tasks. OGB provides fixed dataset splits, a standardized evaluation procedure, and a public leaderboard. The ogbn-arxiv task uses a time-based split, where papers before 2017 are for training, 2017–2018 for validation, and after 2018 for testing. This is more realistic than a random split because it prevents the model from using information about future papers during training.

III. Methodology

This section describes how each model works mathematically and explains why GPS requires a different training strategy at the ogbn-arxiv scale.

A. Graph Convolutional Network Layer

Let $\hat{A} = A + I$ be the adjacency matrix with self-loops added, $\hat{D}$ the corresponding diagonal degree matrix, $H^{(l)} \in \mathbb{R}^{N \times d_l}$ the node feature matrix at layer l, $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ a learnable weight matrix, and σ a nonlinear activation function (ReLU in our case). The GCN update rule for a single layer is

$$H^{(l+1)} = \sigma\Big(\hat{D}^{-\frac{1}{2}}\,\hat{A}\,\hat{D}^{-\frac{1}{2}}\,H^{(l)}\,W^{(l)}\Big)\,. \tag{1}$$

The $\hat{D}^{-1/2}\hat{A}\hat{D}^{-1/2}$ term normalizes the aggregation by both the degree of the neighbor and the degree of the target node. This prevents high-degree nodes from dominating the aggregation and keeps the scale of representations stable across layers. Because the normalization matrix can be precomputed once from the graph structure, the forward pass reduces to a sparse matrix multiply, which is why GCN is so fast in practice. In our implementation we use 3 GCN layers with hidden dimension 256 and dropout $p = 0.5$ between layers.

B. Graph Attention Network Layer

GAT introduces learnable attention weights instead of the fixed normalization used by GCN. Let $\mathbf{h}_i, \mathbf{h}_j \in \mathbb{R}^d$ be the current representations of nodes i and j, $W \in \mathbb{R}^{d' \times d}$ a shared linear transformation, and $\mathbf{a} \in \mathbb{R}^{2d'}$ the attention weight vector. The attention score from neighbor j to node i is computed as

$$\alpha_{ij} = \mathrm{softmax}_j\big(\mathrm{LeakyReLU}\big(\mathbf{a}^T[W\mathbf{h}_i \,\|\, W\mathbf{h}_j]\big)\big)\,, \tag{2}$$

where $\|$ denotes concatenation and $\mathrm{softmax}_j$ is taken over all neighbors of i. The updated node representation is then $\mathbf{h}'_i = \sigma\Big(\sum_{j\in\mathcal{N}(i)} \alpha_{ij} W\mathbf{h}_j\Big)$. Using K attention heads means running K independent versions of this process and concatenating the results, which gives a richer representation. In our setup, we use 3 GAT layers with $K = 8$ heads and per-head hidden dimension 128, giving a total hidden width of 1024. The output layer uses a single head, and dropout $p = 0.3$ is applied to both the features and the attention scores.

C. GPS Layer: Combining Local and Global Attention

A GPS layer [3] runs two separate branches on the same node representations and then combines them. The local branch is a standard message-passing network (we use GCNConv) that aggregates from immediate neighbors. The global branch is a full Transformer self-attention that allows every node in the current subgraph to attend to every other node. If we call the outputs of these two branches $\mathbf{h}_v^{\mathrm{MPNN}}$ and $\mathbf{h}_v^{\mathrm{Attn}}$ for node v, then the GPS layer update is

$$\mathbf{h}_v^{(l+1)} = \mathrm{MLP}\Big(\mathbf{h}_v^{\mathrm{MPNN},(l)} + \mathbf{h}_v^{\mathrm{Attn},(l)}\Big)\,, \tag{3}$$

where both branches use layer normalization and residual connections. An important addition is Laplacian Positional Encoding (LapPE): the k smallest non-trivial eigenvectors of the graph Laplacian are appended to each node's feature vector before the first layer. This gives the Transformer a way to tell nodes apart based on their position in the graph, which it cannot do on its own. In our model we use $k = 16$ eigenvectors, 4 GPS layers, and hidden dimension 256 with $K = 8$ attention heads.

D. Scalability Constraint: ClusterLoader for GPS

Running full self-attention over all $N = 169{,}343$ nodes at once requires $\mathcal{O}(N^2)$ memory, which is not feasible on standard GPU hardware. To handle this, we use ClusterLoader: the graph is split into 32 balanced clusters using METIS, and each training step processes one cluster at a time. This means the global attention in Equation (3) only runs over roughly 5,300 nodes per cluster instead of the full graph. As we discuss in Section V, this significantly limits how much GPS can benefit from its global attention component.

IV. Implementation

A. Data

The ogbn-arxiv dataset comes from the PyTorch Geometric OGB interface and represents citation relationships between computer-science papers on arXiv. Table I summarizes the key dataset statistics. The split is time-based: papers published before 2017 are in the training set, papers from 2017–2018 are in the validation set, and papers after 2018 are in the test set. Node features are 128-dimensional word2vec embeddings averaged over each paper's title and abstract. The class distribution is quite imbalanced: cs.CV has 27,321 nodes while cs.GL has only 29, so models that do well on average accuracy need to handle rare categories too.

TABLE I: Dataset Characteristics of ogbn-arxiv

Property	Value
Dataset	ogbn-arxiv
Nodes	169,343
Edges (directed)	1,166,243
Node features	128
Classes	40
Train nodes	90,941
Validation nodes	29,799
Test nodes	48,603
Mean undirected degree	13.67

B. Model Architecture and Training Setup

All models were implemented in Python 3.11 using PyTorch 2.x and PyTorch Geometric, and training was done on a Google Colab instance with an NVIDIA GPU. Hyperparameters were set through YAML config files, and the key settings for each model are described below.

GCN uses 3 layers with hidden dimension 256, ReLU activations, and dropout $p = 0.5$. It is trained with Adam (learning rate 10^{-3}, weight decay 5×10^{-4}) for up to 500 epochs with early stopping after 50 epochs without validation improvement. The full 169K-node graph is loaded onto the GPU for every training step.

GAT uses 3 layers with 8 attention heads and per-head hidden dimension 128. Dropout $p = 0.3$ is applied to both the node features and attention weights. Adam is used with learning rate 3×10^{-3} and no weight decay, with the same full-batch setup and patience of 80 epochs.

GPS uses 4 layers with hidden dimension 256 and 8 attention heads. Before the first layer, the 128-dimensional node feature and the 16-dimensional LapPE vector are concatenated and projected to dimension 256. ClusterLoader splits the graph into 32 roughly equal clusters. We use Adam with learning rate 10^{-3} and weight decay 10^{-4}, a ReduceLROnPlateau scheduler (factor 0.5, patience 20, minimum 10^{-5}), and gradient clipping at norm 1.0. Training runs for up to 500 epochs with patience 100. The Laplacian eigenvectors are precomputed once and cached on disk so they do not need to be recomputed every run.

C. Evaluation Metrics

We follow the standard OGB evaluation protocol, which uses the official OGB evaluator to compute top-1 node classification accuracy on the test set. The reported test accuracy comes from the model checkpoint that had the best validation accuracy during training. We also compute per-class accuracy by applying the best checkpoint to all nodes and measuring accuracy separately within each of the 40 subject categories. This is important for answering the second research question about whether GPS helps interdisciplinary classes, since top-1 overall accuracy can hide differences at the class level.

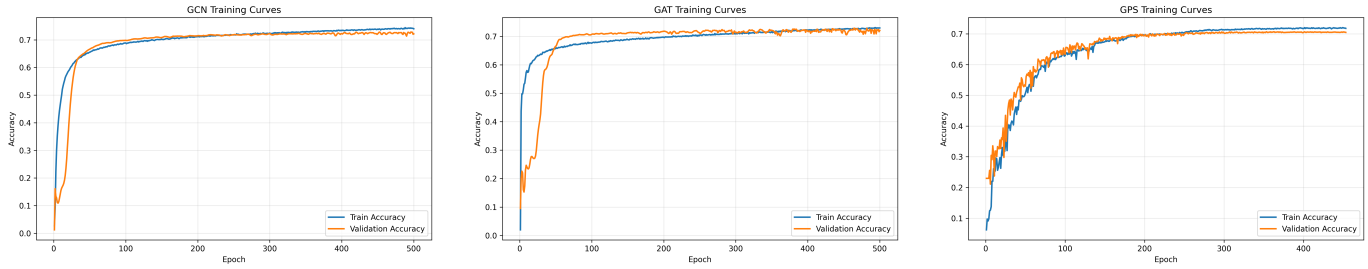
V. Results

A. Overall Test Accuracy

Table II shows the test accuracy, validation accuracy, parameter count, and approximate training time per epoch for each model. GCN achieved the best test accuracy at 72.00% while also having the fewest parameters (110,120) and the shortest training time per epoch (about 2.4 seconds). This confirms that GCN is a very strong and efficient baseline. GAT came in close at 71.75%, but its parameter count is about 12× larger than GCN's (1,362,040 parameters). Given that the accuracy difference is less than 0.3 percentage points, it is not clear that the added complexity of attention is worth it here. GPS had the lowest test accuracy at 68.97%, about 3 percentage points below GCN, and also took the longest to train at around 34 seconds per epoch due to the overhead of mini-batch processing with ClusterLoader.

TABLE II: Performance Comparison on ogbn-arxiv

Model	Test Acc. (%)	Val Acc. (%)	Macro Acc. (%)	# Params
GCN	72.00	72.84	48.36	110K
GAT	71.75	72.85	51.26	1.36M
GPS	68.97	70.67	47.95	2.42M



(a) GCN (500 epochs) (b) GAT (500 epochs) (c) GPS (453 epochs, early stopped)

Fig. 1: Training and validation accuracy over epochs for each model. GCN and GAT converge to similar validation levels around 72–73%, while GPS levels off about 2 percentage points lower. The gap reflects the reduced attention context each cluster has under ClusterLoader.

Figure 1 shows how validation accuracy changed over training for all three models. GCN and GAT follow similar paths and both settle near 72–73%. GPS rises more slowly and triggers early stopping at epoch 453, which suggests it ran out of room to improve within the mini-batch setting. The learning rate scheduler also reduced the learning rate several times during GPS training in response to flat validation accuracy, which likely prevented further improvement.

B. *Per-Class Accuracy and Cross-Domain Analysis*

Even though GPS had lower overall accuracy, it actually improved over GCN on some individual classes. Table III shows the five classes where GPS gained the most and the five where it lost the most compared to GCN. The biggest gains were in cs.NE (+10.2 pp), cs.AR (+8.0 pp), cs.SD (+7.6 pp), cs.IR (+6.6 pp), and cs.SE (+5.8 pp). These are all relatively small or cross-disciplinary categories. The biggest losses were in cs.LG (−12.6 pp), cs.DB (−7.9 pp), cs.DM (−6.7 pp), cs.LO (−6.7 pp), and cs.SC (−5.6 pp), which are either very large categories or ones with tight internal citation structure. We computed the Pearson correlation between each class's cross-domain citation ratio and the GPS-minus-GCN accuracy change and got $r = 0.12$. This is a weak positive relationship, meaning classes that cite more broadly across fields do tend to benefit slightly more from GPS.

TABLE III: Largest Per-Class GPS Gains and Losses Relative to GCN

Class	Label	Δ Acc (pp)
cs.NE	Neural and Evolutionary Computing	+10.2
cs.AR	Hardware Architecture	+8.0
cs.SD	Sound	+7.6
cs.IR	Information Retrieval	+6.6
cs.SE	Software Engineering	+5.8
cs.LG	Machine Learning	−12.6
cs.DB	Databases	−7.9
cs.DM	Discrete Mathematics	−6.7
cs.LO	Logic in CS	−6.7
cs.SC	Scientific Computing	−5.6

One category we were specifically interested in was cs.HC (Human-Computer Interaction), which has the fourth-highest cross-domain citation ratio among all 40 classes at 69.5%. On cs.HC, GPS reached 24.8% per-class accuracy, compared to 23.6% for GCN and 22.5% for GAT. The improvement is small but consistent with the idea that global context helps nodes that connect to many different research areas.

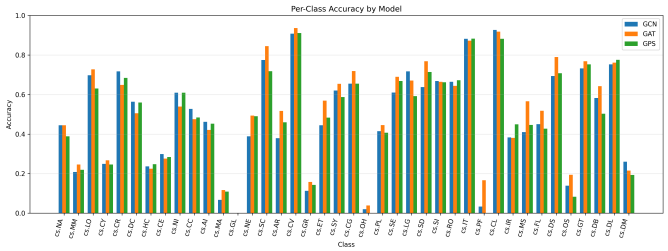


Fig. 2: Per-class accuracy for GCN, GAT, and GPS across all 40 ogbn-arxiv subject categories. GPS underperforms on large, self-citing categories like cs.LG and cs.CV, but narrows the gap or outperforms GCN on smaller cross-disciplinary ones.

Figure 2 shows the full per-class accuracy breakdown across all 40 categories. The macro-averaged per-class accuracy was 51.26% for GAT, 48.36% for GCN, and 47.95% for GPS. GAT having the best macro average even while slightly trailing GCN in overall accuracy suggests its attention mechanism helps more on smaller and rarer categories, which contribute equally to the macro average.

C. *Analysis of GPS Underperformance*

The result that GPS performs worse than GCN was not what we expected going in, since the GPS paper [3] reports roughly 75% test accuracy on ogbn-arxiv using full-graph attention on dedicated hardware. There are a few reasons why we think our result came out differently. First, and most importantly, the ClusterLoader approach limits the global attention in Equation (3) to operate within clusters of about 5,300 nodes rather than across the full 169K-node graph. This takes away most of the advantage that GPS has over message-passing models. Second, GPS has 2.42M parameters compared to GCN's 110K, so it needs more training examples and more epochs to fully converge, and our 453-epoch run

with aggressive early stopping may not have been enough. Third, the ReduceLROnPlateau scheduler reduced the learning rate multiple times in response to noisy mini-batch validation accuracy, which may have stopped training too early. Taken together, these factors suggest that GPS's advantage over GCN is not automatic and depends heavily on having enough compute to run full-graph attention.

VI. Conclusion

This project compared GCN, GAT, and GPS on the ogbn-arxiv node classification task. The main takeaway is that under the compute constraints of a standard GPU environment, GCN was the best overall option: it reached 72.00% test accuracy with only 110K parameters and trained in about 2.4 seconds per epoch. GAT was close at 71.75% but cost 12× more in parameters with no clear accuracy gain. GPS came in last at 68.97%, which was surprising given that it is supposed to be the most expressive of the three. The reason comes down to the ClusterLoader mini-batching that was needed to fit the model on GPU: by restricting self-attention to within-cluster context, it largely removes the benefit that makes GPS better than GCN in the first place.

The bigger lesson from this is that there is a real tension between expressiveness and scalability in graph transformers. GPS is designed to model long-range dependencies through global attention, but large graphs require breaking the graph into pieces during training, and doing so defeats the purpose of global attention. The original GPS paper achieved 75% on ogbn-arxiv because it had access to larger hardware that could support full-graph attention. For someone training on a standard GPU, GCN or GAT may actually be the better choice.

On the per-class side, we did find a weak positive signal that GPS helps categories with high cross-domain citation ratios, such as cs.HC, cs.NE, and cs.AR. This is consistent with the idea that global attention is most useful when a node draws from many different areas of the graph. One direction for future work would be to explore hybrid approaches where GPS-style global attention is only applied to a subset of nodes identified as cross-domain, which might recover some of the accuracy gains without the full computational cost. Other interesting directions include applying these models to heterogeneous graphs that have multiple node and edge types, or extending the comparison to link prediction tasks.

References

[1] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017. [Online]. Available: https://arxiv.org/abs/1609.02907

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018. [Online]. Available: https://arxiv.org/abs/1710.10903

[3] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, "Recipe for a general, powerful, scalable graph transformer," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. [Online]. Available: https://arxiv.org/abs/2205.12454